

Aula 19: Arquiteturas de Normalizing Flows

Coupling, Autoregressão, Glow, MADE, IAF/MAF

Disciplina de Aprendizagem Profunda

Agenda da Aula

- Recapitulando Normalizing Flows
- Representações internas em deep learning
- Coupling flows: limitações e melhorias
- Glow: 1×1 convolução invertível
- Flows autoregressivos: MAF e IAF
- MADE: redes mascaradas
- Outras famílias de flows
- Design prático de flows
- Exercício de implementação: MAF

Recapitulando

- Flows transformam $z \sim p_Z$ em $x \sim p_X$
- Bijeções + Jacobianas tratáveis
- Coupling layers:

$$z_1 = x_1, \quad z_2 = (x_2 - m(x_1)) \exp(-\alpha(x_1)).$$

- Log-likelihood:

$$\log p_X(x) = \log p_Z(z) + \sum \log |\det J|.$$

- Experimento: TwoSpirals

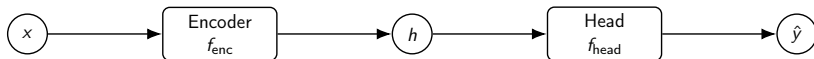
Deep Learning e Representação Interna

- Toda rede profunda possui duas partes:

$$h = f_{\text{enc}}(x), \quad \hat{y} = f_{\text{head}}(h).$$

- h = representação de alto nível.
- Essa representação pode ser usada em:
 - classificação (supervisionado)
 - pré-treinamento (auto-supervisionado)
 - modelagem generativa

Representação como Encoder + Cabeça



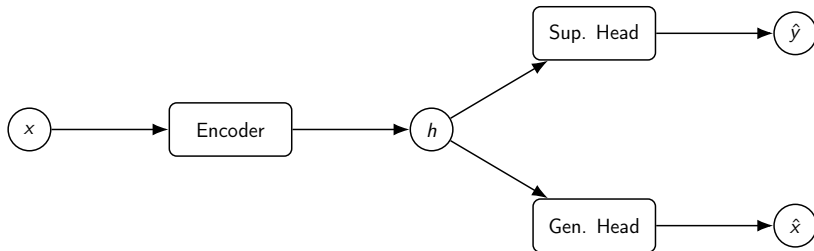
Três Regimes Fundamentais

- **Supervisionado:** aprende $p(y \mid x)$
- **Auto-supervisionado:** aprende estrutura de x via tarefas sintéticas
- **Generativo:** aprende $p(x)$ ou $p(x, y)$
- Todos aprendem uma **representação interna** de x .

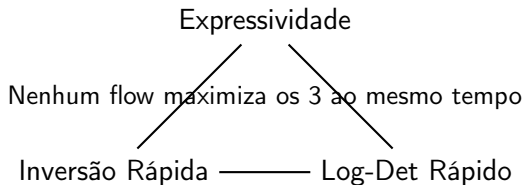
Reuso de Representações em Tarefas Múltiplas

- Treine encoder em um regime; use-o em outro.
- Exemplo:
 - pré-treinamento auto-supervisionado;
 - usar h em classificação;
 - usar h para parametrizar modelo generativo.

Encoder Único, Múltiplas Cabeças



O Espaço de Design de Flows



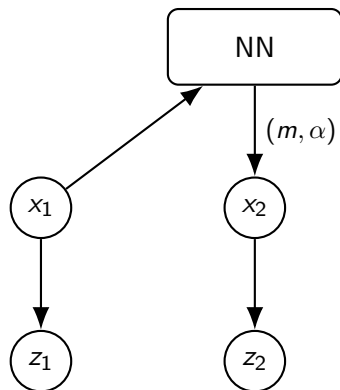
Coupling Layers: Revisão

- Divisão de variáveis: (x_A, x_B) .
- x_A fica fixo; x_B é transformado:

$$z_B = (x_B - m(x_A))e^{-s(x_A)}.$$

- Jacobiano triangular: fácil log-det.

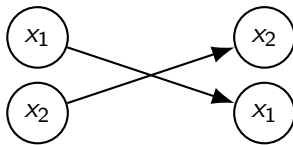
Diagrama de Coupling Layer



Limitações de Coupling Simples

- Cada camada só transforma metade das variáveis.
- Sem permutações, fica pouco expressivo.
- Precisamos de mecanismos de mistura:
 - reverse
 - shuffle
 - 1×1 conv invertível

Mistura de Variáveis via Permutações



Exemplo: Reverse

Glow: Arquitetura em Alto Nível

- Cada etapa contém:
 - 1 ActNorm
 - 2 Convolução 1×1 invertível
 - 3 Coupling layer
- Melhor mistura e maior estabilidade.

ActNorm: Normalização Aprendida

- Inicializada pelos primeiros batches de dados
- Evita problemas de escalas ruins no início do treino
- Substitui batchnorm nos flows

Convolução 1×1 Invertível

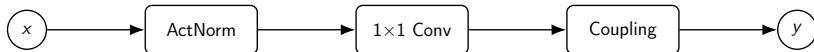
$$y = Wx, \quad W \in GL(D).$$

- Permite mistura linear total
- Log-det = $\log |\det W|$ constante por pixel
- Implementação eficiente via decomposição LU

TikZ: Mistura Linear

$$\begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

Diagrama Glow (1 bloco)



Vantagens do Glow

- Mistura mais rica que reverse/shuffle
- Jacobiano simples
- Escalável para imagens grandes
- Mais estável que RealNVP original

Para Onde Vamos Agora?

- Até agora: flows baseados em **coupling**.
- Agora: flows baseados em **autoregressão**.
- Permitem modelagem muito mais expressiva.

Autoregressive Flows (MAF e IAF)

Princípio de Autoregressão

- Fatorizamos a distribuição:

$$p(x) = \prod_{i=1}^D p(x_i \mid x_{1:i-1}).$$

- Cada variável depende apenas das anteriores.
- Essa estrutura leva a flows altamente expressivos.

Transformações Autoregressivas

Em um flow autoregressivo:

$$z_i = \frac{x_i - m_i(x_{1:i-1})}{\exp(\alpha_i(x_{1:i-1}))}.$$

- m_i e α_i são parametrizados por uma rede neuronal.
- Jacobiano triangular $\rightarrow$ log-det eficiente.

Jacobiano Triangular

A jacobiana é triangular inferior:

$$J_{ij} = \frac{\partial z_i}{\partial x_j} = \begin{cases} * & j \leq i \\ 0 & j > i \end{cases}.$$

Assim:

$$\log |\det J| = \sum_{i=1}^D -\alpha_i(x_{1:i-1}).$$

- O cálculo de log-densidade é $O(D)$.
- Ideal para **density estimation**.

MAF vs IAF: Diferença Estrutural

■ MAF (Masked Autoregressive Flow):

- $z \leftarrow x$ é autoregressivo
- log-likelihood rápido
- amostragem lenta

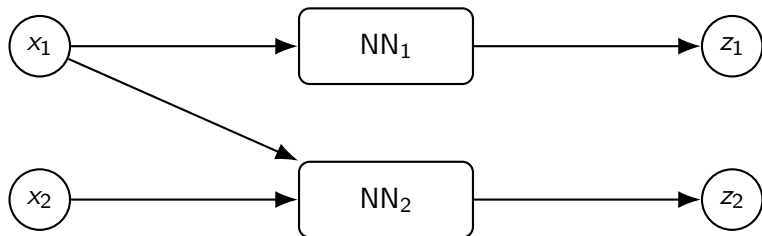
■ IAF (Inverse Autoregressive Flow):

- $x \leftarrow z$ é autoregressivo
- amostragem rápida
- log-likelihood lenta

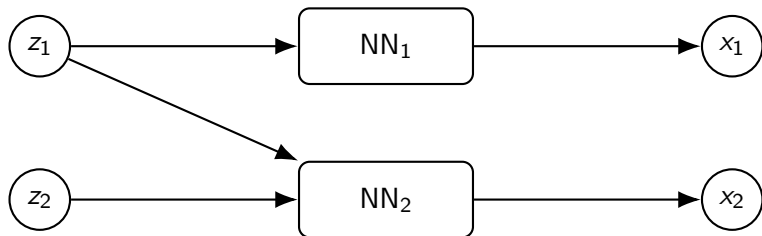
Comparação MAF vs IAF

Flow	Amostragem	Log-likelihood	Uso típico
MAF	Lenta	Rápida	Density estimation
IAF	Rápida	Lenta	Variational inference

MAF (2D) – Diagrama



IAF (2D) – Diagrama



Resumo Matemático (MAF)

$$z_i = \frac{x_i - m_i(x_{<i})}{\exp(s_i(x_{<i}))}.$$

$$\log |\det J| = - \sum_i s_i(x_{<i}).$$

$$\log p_X(x) = \log p_Z(z) + \log |\det J|.$$

Quando Usar Flows Autoregressivos?

- Quando densidade é prioridade
- Quando D não é enorme (embora MADE escale bem)
- Para modelagem de dados tabulares
- Para enriquecer VAEs (IAF na posterior)

MADE: Máscaras para Autoregressão

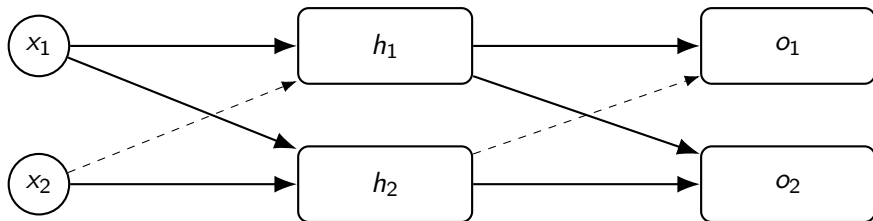
- Impõe autoregressão em MLPs usando máscaras binárias.
- Cada neurônio recebe um índice que determina quem ele pode “ver”.
- Garante: saída i não depende de x_j com $j \geq i$.

Máscaras Autoregressivas

$$W_{\text{mascarado}} = W \odot M.$$

- $M_{ij} = 1$ se conexão permitida.
- $M_{ij} = 0$ se proibida pela ordem autoregressiva.

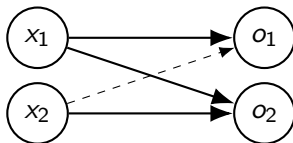
Exemplo MADE (2 Variáveis)



Matrizes de Máscara (Exemplo)

$$M_{\text{in} \rightarrow \text{hid}} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}, \quad M_{\text{hid} \rightarrow \text{out}} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}.$$

Gráfico de Dependências Autoregressivas



De MADE $\rightarrow$ MAF $\rightarrow$ IAF

- MADE: rede autoregressiva mascarada.
- MAF: aplica MADE para mapear $x \rightarrow z$.
- IAF: aplica MADE para mapear $z \rightarrow x$.
- Todas usam a mesma ideia de dependência ordenada.

Continuous Normalizing Flows (CNF)

- Em vez de bijeções discretas, usamos um fluxo contínuo:

$$\frac{dz(t)}{dt} = f_{\theta}(z(t), t).$$

- A solução da EDO define um difeomorfismo:

$$z(T) = \Phi_{\theta}(z(0)).$$

- Conecta diretamente flows e Neural ODEs.

Neural ODE como Transformação Invertível

- Se f_θ for suave $\rightarrow$ solução é invertível.
- Amostragem: integrar de $0 \rightarrow T$.
- Inversão: integrar de $T \rightarrow 0$.
- Flow contínuo entre base e dados.

Mudança de Variáveis Contínua (FFJORD)

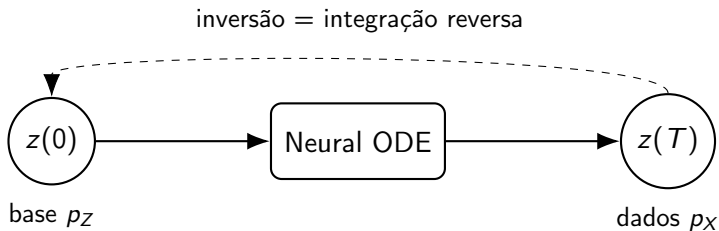
$$\frac{d}{dt} \log p(z(t)) = -\text{Tr} \left(\frac{\partial f_{\theta}}{\partial z}(z(t), t) \right).$$

Integrando:

$$\log p_X(x) = \log p_Z(z(0)) - \int_0^T \text{Tr} \left(\frac{\partial f_{\theta}}{\partial z} \right) dt.$$

- FFJORD usa estimadores de traço (Hutchinson).
- Calcula densidade + transformação via ODE solver.

Fluxo Contínuo no Tempo (TikZ)



Resumo das Famílias de Flows

Família	Vantagens	Desvantagens
Coupling Glow (1×1) MAF IAF Splines CNF (FFJORD)	rápido amostrar mistura forte LL rápido amostragem rápida muito expressivos contínuo e flexível	mistura limitada custo linear adicional amostragem lenta LL lenta complexidade maior custo ODE alto

Design Prático de Normalizing Flows

- Arquitetura de cada bloco (acoplamento, autoregressivo, linear, spline)
- Mistura de variáveis (permutação, conv 1×1 , LU)
- Profundidade (n° de camadas)
- Estabilidade numérica
- Eficiência: custo de amostragem e de log-likelihood

Design da Rede Parametrizadora

- Redes usadas para prever (m, α) ou splines:
 - MLPs
 - CNNs
 - Redes residuais
- Boas práticas:
 - camadas residuais ajudam estabilidade
 - clamp/tanh em α evita explosão
 - normalização inicial (ActNorm ou camada customizada)

Estratégias de Mistura Entre Camadas

- **Reverse:** simples, barato, melhora acoplamento
- **Shuffle:** aleatoriza a ordem dos canais
- **1×1 Conv Invertível:** mistura linear completa
- **LU Factorization:** log-det barato
- **Multi-scale:** divide canais ao longo da rede

Diagnóstico Durante Treinamento

- Monitorar $\log p(x)$
- Checar distribuição das escalas α
- Observar o sinal dos log-dets
- Plotar amostras progressivamente ao longo do treinamento
- Detectar colapso:
 - escalas muito grandes/pequenas
 - saturação do MLP
 - permutações insuficientes

10 Regras Práticas de Design de Flows

- 1 Misture variáveis com frequência.
- 2 Use clamping/tanh nas escalas.
- 3 Prefira redes residuais se possível.
- 4 Não exagere na profundidade antes de testar versões menores.
- 5 Comece com affine coupling e depois teste splines.
- 6 Monitore log-dets.
- 7 Aumente progressivamente a capacidade da rede.
- 8 Para D pequeno, considere linear invertível.
- 9 Para D alto (imagens), prefira multi-scale.
- 10 Lembre: sampling vs likelihood define MAF ou IAF.

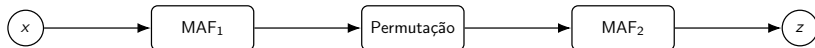
Exercício de Implementação: MAF

Objetivo: implementar um fluxo autoregressivo simples para TwoSpirals.

Tarefas:

- 1 Implementar `MAF.forward(x)`
- 2 Implementar `MAF.inverse(z)`
- 3 Criar uma máscara MADE simples para 2D
- 4 Treinar o fluxo e comparar com o RealNVP da Aula 17

Estrutura do Flow do Exercício (TikZ)



Comparação Esperada dos Resultados

- MAF $\rightarrow$ melhor ajuste de log-likelihood
- Coupling $\rightarrow$ melhor velocidade de amostragem
- Visualizar hist2d das amostras:
 - qualidade das espirais
 - distância entre braços
- Analisar estabilidade do treinamento

Resumo da Aula

- Coupling flows: rápidos, simples, mas misturam pouco
- Glow: mistura aprendida via 1×1 conv
- Autoregressão: MAF e IAF, altamente expressivos
- MADE: máscaras para impor autoregressão
- CNFs (Neural ODEs): fluxos contínuos (FFJORD)
- Princípios de design: mistura, estabilidade, profundidade
- Exercício: implementar MAF e comparar com RealNVP

Próxima Aula: Variational Autoencoders

- ELBO e otimização variacional
- Reparameterization trick
- Encoder probabilístico
- Decoder generativo
- Como VAEs conectam com flows (IAF)

Até a próxima!